

 FORMATO PARA ELABORACIÓN DE TALLERES ESTUDIANTILES	Departamento: Ciencias de la Computación
	Carrera: Ingeniería de Software
	Asignatura: Análisis y Diseño de Software NRC: 30723
	Apellidos y nombres del estudiante: Caiza Carlos, Kerly Chiriboga, Alexander Villacrés

BOCATTO V2.0 — Plan de Pruebas Unitarias

Identificador del documento: TP-BOCATTO-UT-001

Norma de referencia: IEEE Std 829-2008 (Standard for Software and System Test Documentation)

Proyecto: BOCATTO V2.0

Tipo de documento: Plan de Pruebas Unitarias

Alcance: Backend (Node.js / Express / Mongoose) y Frontend (SPA en JavaScript puro)

TABLA DE CONTENIDOS

TABLA DE CONTENIDOS.....	1
1. IDENTIFICADOR DEL PLAN DE PRUEBAS.....	1
2. INTRODUCCIÓN.....	1
2.1 Propósito.....	1
2.2 Antecedentes.....	2
2.3 Módulos del proyecto bajo prueba.....	2
2.4 Definiciones, acrónimos y abreviaturas.....	3
2.5 Referencias.....	3
3. ELEMENTOS A PROBAR.....	4
3.1 Elementos del backend (módulos Node.js, ES modules).....	4
3.2 Elementos del frontend (Vanilla JS, ES modules).....	5
3.3 Elementos excluidos de las pruebas unitarias (fuera de alcance).....	5
4. CARACTERÍSTICAS A PROBAR.....	5
4.1 Reglas de validación (BE y FE).....	6
4.2 Lógica de dominio — patrón Builder y Director (BE).....	6
4.3 Patrón Prototype (BE).....	6
4.4 DTO / Mapper (BE).....	7
4.5 Reglas de negocio de la capa de servicio (BE).....	7
4.6 Middleware (BE).....	7

4.7 Utilidad HTTP / respuesta (BE).....	7
4.8 Frontend — clases utilitarias (FE).....	8
4.9 Frontend — servicios sobre fetch (FE)	8
4.10 Frontend — bus de eventos y helpers de UI (FE)	8
5. CARACTERÍSTICAS QUE NO SE PROBARÁN.....	8
6. ENFOQUE DE PRUEBAS.....	9
6.1 Niveles de prueba	9
6.2 Técnica de prueba	9
6.3 Framework de pruebas	10
6.4 Convención de estructura de pruebas.....	10
6.5 Patrón de diseño de pruebas.....	11
6.6 Estrategia de mocks.....	11
6.7 Objetivos de cobertura de código	11
6.8 Gestión de datos de prueba.....	12
6.9 Herramientas e Integración Continua.....	12
7. CRITERIOS DE APROBACIÓN/FALLO.....	12
7.1 Aprobación/Fallo por caso de prueba.....	12
7.2 Aprobación/Fallo por módulo	12
7.3 Aprobación/Fallo por componente (backend / frontend)	12
7.4 Aprobación/Fallo global	12
8. CRITERIOS DE SUSPENSIÓN Y REQUISITOS DE REANUDACIÓN.....	13
8.1 Criterios de suspensión (cuándo detener la fase).....	13
8.2 Requisitos de reanudación	13
9. ENTREGABLES DE LAS PRUEBAS	13
11. NECESIDADES DEL ENTORNO	15
11.1 Hardware	15
11.2 Software	15
11.3 Red	15
11.4 Archivos de configuración a añadir.....	15
12. CALENDARIO.....	15
13. RIESGOS Y CONTINGENCIAS.....	16
13.1 Riesgos del proyecto.....	16
13.2 Esquema de severidad de defectos	17
14. APROBACIONES	17
APÉNDICE A — ESPECIFICACIONES DE PRUEBAS UNITARIAS DEL BACKEND	17
A.1 BE-01 — ApiError (backend/src/utls/ApiError.js)	17

1. IDENTIFICADOR DEL PLAN DE PRUEBAS

Campo	Valor
ID del documento	TP-BOCATTO-UT-001
Título	BOCATTO V2.0 — Plan de Pruebas Unitarias
Versión	1.0
Estado	Borrador
Autor	Carlos Caiza
Fecha de emisión	2026-07-08
Norma	IEEE Std 829-2008
Planes relacionados	n/a

2. INTRODUCCIÓN

2.1 Propósito

Este documento define el plan, el enfoque, los recursos y el calendario para las actividades de pruebas unitarias del sistema de gestión de restaurante BOCATTO V2.0. Está pensado para servir como plano de referencia del proceso de pruebas: el equipo que ejecute las pruebas deberá seguirlo como la referencia autorizada para alcance, técnica, criterios y entregables.

2.2 Antecedentes

BOCATTO V2.0 es una aplicación web full-stack que respalda las operaciones de un restaurante: administración de usuarios y roles, registro de clientes, gestión de mesas, menú, construcción de pedidos, flujo de cocina, caja, promociones, tableros (dashboards) y reportes. Además soporta un flujo público de pedidos por QR.

El sistema cuenta con dos componentes desplegables de forma independiente:

- **Backend:** Node.js 20+, Express 5, Mongoose 8, MongoDB, autenticación con JWT, Socket.IO.
- **Frontend:** SPA en JavaScript puro (sin framework de build), estructura modular (servicios, controladores, vistas, validadores, utilidades), plantillas HTML, CSS, cliente de Socket.IO.

2.3 Módulos del proyecto bajo prueba

#	Módulo	Archivos backend	Archivos frontend
1	Autenticación	services/AuthService.js, middlewares/authMiddleware.js, middlewares/roleMiddleware.js	services/authService.js, utils/Auth.js, utils/storage.js
2	Usuarios	services/UsuarioService.js, validators/UsuarioValidator.js, dto/UsuarioDTO.js, mappers/UsuarioMapper.js	services/usuarioService.js, validators/UsuarioValidator.js, views/UsuarioView.js
3	Clientes	services/ClienteService.js, validators/ClienteValidator.js	services/clienteService.js, validators/ClienteValidator.js
4	Mesas	services/MesaService.js, validators/MesaValidator.js, dto/MesaDTO.js, mappers/MesaMapper.js	services/mesaService.js, validators/MesaValidator.js
5	Menú	services/MenuService.js, validators/MenuValidator.js, patterns/prototype/MenuPrototype.js	services/menuService.js, validators/MenuValidator.js

6	Pedidos	services/PedidoService.js, validators/PedidoValidator.js, patterns/builder/PedidoBuilder.js, patterns/builder/DirectorPedido.js, dto/PedidoDTO.js	services/pedidoService.js, validators/PedidoValidator.js, utils/EstadoPedido.js
7	Cocina	services/CocinaService.js	services/cocinaService.js, views/CocinaView.js
8	Caja	services/CajaService.js	services/cajaService.js, views/CajaView.js
9	Promociones	services/PromocionService.js, validators/PromocionValidator.js, patterns/prototype/PromocionPrototype.js	services/promocionesService.js, views/PromocioneView.js
10	Reportes	services/ReporteService.js	services/reporteService.js, views/ReporteView.js
11	Dashboard	services/DashboardService.js	services/dashboardService.js, views/DashboardView.js
12	Utilidades comunes	utils/ApiError.js, utils/ApiResponse.js	utils/EventBus.js, utils/Toast.js, utils/Modal.js, utils/Form.js, utils/Loader.js, utils/AppState.js, utils/router.js, utils/layout.js, utils/socket.js, utils/Alert.js

2.4 Definiciones, acrónimos y abreviaturas

Término	Significado
SUT	System Under Test (Sistema Bajo Prueba)
DUT	Design (or class) Under Test (Diseño o clase bajo prueba)
DTO	Data Transfer Object (Objeto de Transferencia de Datos)
API	Application Programming Interface
JWT	JSON Web Token
BC	Boundary Condition (Condición de frontera, p. ej. vacío, nulo, mín., máx.)
EP	Equivalence Partition (Partición de equivalencia)
BVA	Boundary Value Analysis (Análisis de valores límite)
AAA	Arrange / Act / Assert (patrón Preparar / Actuar / Afirmar)
MUT	Module Under Test (Módulo bajo prueba)
CLI	Command Line Interface (Interfaz de línea de comandos)
SPA	Single-Page Application (Aplicación de una sola página)

2.5 Referencias

- IEEE Std 829-2008 — Standard for Software and System Test Documentation.
- especificación de Requisitos de Software (SRS) v1.0.
1. Elicitacion/1.1 SRS/G2_30723_SRS_ADS_V1.0.pdf
- Historias de Usuario y Backlog.
1. Elicitacion/1.4 Historias de Usuario/, 1. Elicitacion/1.6 Backlog/
- Documento de Diseño de Arquitectura v1 y v2.
2. Diseños/1.1 Diseño de Arquitectura/Arquitectura_Propuesta_G2_V1.pdf,
Arquitectura_Propuesta_G2_V2.pdf
- atrones de Diseño aplicados.
2. Diseños/1.0 Patron de diseño/
- Diagramas de clases, componentes y ER.
2. Diseños/1.3 Diagramas de Clase/,
2. Diseños/1.4 Diagramas de componentes/,
2. Diseños/1.5 Diagramas ER/
- Diagramas de casos de uso de análisis.
0. Analisis/G2_30723_GUIA{1,2}_ADS_V{1..4}.pdf
- ECMAScript 2022 (ES13) — Language Specification.
<https://tc39.es/ecma262/>

- Jest — JavaScript testing framework (backend).
<https://jestjs.io/docs/getting-started>
- Vitest — Testing framework con soporte nativo para ESM y jsdom (frontend).
<https://vitest.dev/guide/>
- jsdom — Implementación en JavaScript del DOM para Node.js.
<https://github.com/jsdom/jsdom>

3. ELEMENTOS A PROBAR

Los siguientes elementos están en el alcance de las pruebas unitarias.

3.1 Elementos del backend (módulos Node.js, ES modules)

Cada elemento es un único archivo fuente salvo que se indique lo contrario. Las importaciones/exportaciones se siguen solo cuando es necesario para instanciar la unidad.

ID	Elemento	Ruta
BE-01	Clase ApiError	backend/src/utls/ApiError.js
BE-02	Clase ApiResponse	backend/src/utls/ApiResponse.js
BE-03	UsuarioValidator	backend/src/validators/UsuarioValidator.js
BE-04	ClienteValidator	backend/src/validators/ClienteValidator.js
BE-05	MesaValidator	backend/src/validators/MesaValidator.js
BE-06	MenuValidator	backend/src/validators/MenuValidator.js
BE-07	PedidoValidator	backend/src/validators/PedidoValidator.js
BE-08	PromocionValidator	backend/src/validators/PromocionValidator.js
BE-09	MesaDTO	backend/src/dto/MesaDTO.js
BE-10	MesaMapper	backend/src/mappers/MesaMapper.js
BE-11	PromocionPrototype	backend/src/patterns/prototype/PromocionPrototype.js
BE-12	MenuPrototype	backend/src/patterns/prototype/MenuPrototype.js
BE-13	Prototype (abstracta)	backend/src/patterns/prototype/Prototype.js
BE-14	PedidoBuilder	backend/src/patterns/builder/PedidoBuilder.js
BE-15	DirectorPedido	backend/src/patterns/builder/DirectorPedido.js
BE-16	MesaService	backend/src/services/MesaService.js (repositorio mockeado)
BE-17	AuthService	backend/src/services/AuthService.js (repositorio mockeado)
BE-18	Fábrica roleMiddleware	backend/src/middlewares/roleMiddleware.js
BE-19	authMiddleware	backend/src/middlewares/authMiddleware.js
BE-20	errorMiddleware	backend/src/middlewares/errorMiddleware.js

3.2 Elementos del frontend (Vanilla JS, ES modules)

ID	Elemento	Ruta
FE-01	Singleton Storage	frontend/utls/storage.js
FE-02	Singleton Auth	frontend/utls/Auth.js
FE-03	Constantes EstadoPedido	frontend/utls/EstadoPedido.js
FE-04	MesaValidator	frontend/validators/MesaValidator.js
FE-05	MenuValidator	frontend/validators/MenuValidator.js
FE-06	PedidoValidator	frontend/validators/PedidoValidator.js
FE-07	ClienteValidator	frontend/validators/ClienteValidator.js
FE-08	UsuarioValidator	frontend/validators/UsuarioValidator.js
FE-09	MesaService (HTTP)	frontend/services/mesaService.js (con stub de fetch)
FE-10	Constante api	frontend/services/api.js
FE-11	Helper Toast	frontend/utls/Toast.js
FE-12	Helper Modal	frontend/utls/Modal.js
FE-13	Helper EventBus	frontend/utls/EventBus.js

Nota: Todas las clases de servicio dependientes de HTTP (FE-09 y el resto de archivos *Service.js) se prueban unitariamente sustituyendo fetch. La red real no se ejerce en este plan.

3.3 Elementos excluidos de las pruebas unitarias (fuera de alcance)

- Implementaciones de repositorios contra MongoDB (pruebas de integración).
- Orquestación HTTP de controladores contra una aplicación Express en ejecución (pruebas de integración).
- Validación y persistencia del esquema Mongoose (prueba de modelo — integración).
- Flujos end-to-end de Socket.IO (pruebas de sistema).
- Flujo end-to-end de pedidos por QR (pruebas de sistema).
- Seeders (herramientas operativas, no son código del SUT).
- Renderizado HTML, CSS y maquetación visual (pruebas de sistema / aceptación).
- Rendimiento, carga y pruebas de seguridad (planes separados).

4. CARACTERÍSTICAS A PROBAR

Las características siguientes se ejercitarán a nivel unitario. Cada una se mapea a uno o más elementos de la Sección 3.

4.1 Reglas de validación (BE y FE)

ID Caract.	Característica	Elementos ejercitados
F-VAL-01	Rechazar nombre vacío o de solo espacios en menú, cliente, promoción, usuario	BE-03...08, FE-04...08
F-VAL-02	Rechazar precio no positivo en menú	BE-06, FE-05
F-VAL-03	Rechazar cantidad no positiva por ítem en pedido	BE-07, BE-14
F-VAL-04	Requerir mesa para canales distintos de TELEFONO	BE-07, BE-14
F-VAL-05	Requerir telefonoEntrega para el canal TELEFONO	BE-07, BE-14, FE-06
F-VAL-06	Requerir clienteNombre para el canal QR	BE-07
F-VAL-07	Forzar el rango de descuento 0..100 para promociones	BE-08
F-VAL-08	Forzar roles válidos (ADMIN, MESERO, COCINA, CAJA) para usuarios	BE-03
F-VAL-09	Forzar longitud mínima de contraseña (≥ 6) para usuarios	BE-03
F-VAL-10	Validar el formato de correo electrónico cuando se proporciona (cliente)	BE-04
F-VAL-11	Forzar numero y capacidad > 0 para mesas	BE-05, FE-04

4.2 Lógica de dominio — patrón Builder y Director (BE)

ID Caract.	Característica	Elementos ejercitados
F-BLD-01	El builder reinicia su estado interno en reset()	BE-14
F-BLD-02	El builder asume canal por defecto MESA y convierte a mayúsculas	BE-14
F-BLD-03	El builder rechaza ítems con cantidad ≤ 0	BE-14
F-BLD-04	El builder rechaza ítems con producto.precio no positivo	BE-14
F-BLD-05	El builder acumula total e ítems	BE-14
F-BLD-06	El builder aplica descuento porcentual mediante aplicarPromocion	BE-14
F-BLD-07	El builder valida los campos requeridos según el canal en validar()	BE-14
F-BLD-08	El builder devuelve una copia profunda en build() y reinicia el estado	BE-14
F-BLD-09	El director orquesta la construcción completa de un pedido (integración de F-BLD-*)	BE-15

4.3 Patrón Prototipe (BE)

ID Caract.	Característica	Elementos ejercitados
F-PRT-01	<i>Prototype.clone()</i> lanza error cuando no se implementa	BE-13
F-PRT-02	<i>PromocionPrototype.clone()</i> devuelve una copia profunda desacoplada	BE-11
F-PRT-03	<i>MenuPrototype.clone()</i> devuelve una copia profunda desacoplada	BE-12

4.4 DTO / Mapper (BE)

ID Caract.	Característica	Elementos ejercitados
F-DTO-01	<i>MesaDTO</i> expone solo la forma pública (id, numero, capacidad, estado)	BE-09
F-MAP-01	<i>MesaMapper.toDTO</i> envuelve una sola entidad	BE-10
F-MAP-02	<i>MesaMapper.toDTOList</i> envuelve un arreglo de entidades	BE-10
F-MAP-03	El mapper maneja un arreglo vacío	BE-10

4.5 Reglas de negocio de la capa de servicio (BE)

ID Caract.	Característica	Elementos ejercitados
F-SRV-01	<i>MesaService.obtenerMesaPorId</i> lanza 404 cuando no existe	BE-16
F-SRV-02	<i>MesaService.cambiarEstado</i> lanza 404 cuando no existe	BE-16
F-SRV-03	<i>MesaService.crearMesa</i> rechaza numero duplicado	BE-16
F-SRV-04	<i>MesaService.actualizarMesa</i> rechaza numero duplicado (que no sea el mismo registro)	BE-16
F-SRV-05	<i>MesaService.eliminarMesa</i> lanza 404 cuando no existe	BE-16
F-SRV-06	<i>AuthService.login</i> rechaza un username desconocido	BE-17
F-SRV-07	<i>AuthService.login</i> rechaza una cuenta bloqueada	BE-17
F-SRV-08	<i>AuthService.login</i> incrementa los intentos fallidos con contraseña incorrecta	BE-17
F-SRV-09	<i>AuthService.login</i> bloquea la cuenta tras 3 intentos fallidos	BE-17
F-SRV-10	<i>AuthService.login</i> reinicia los intentos y devuelve un JWT en caso de éxito	BE-17

4.6 Middleware (BE)

ID Caract.	Característica	Elementos ejercitados
F-MID-01	<i>roleMiddleware</i> retorna 401 cuando falta req.usuario	BE-18
F-MID-02	<i>roleMiddleware</i> retorna 403 cuando el rol no está en la lista permitida	BE-18
F-MID-03	<i>roleMiddleware</i> llama a next() cuando el rol es permitido	BE-18
F-MID-04	<i>authMiddleware</i> rechaza solicitudes sin encabezado Authorization o con uno inválido	BE-19
F-MID-05	<i>authMiddleware</i> carga req.usuario a partir de un JWT válido	BE-19
F-MID-06	<i>errorMiddleware</i> renderiza el sobre estándar de error	BE-20

4.7 Utilidad HTTP / respuesta (BE)

ID Caract.	Característica	Elementos ejercitados
F-UTL-01	<i>ApiError</i> extiende Error y expone status	BE-01
F-UTL-02	<i>ApiError.status</i> vale 500 por defecto	BE-01
F-UTL-03	<i>ApiResponse.success</i> retorna JSON con success: true, message, data	BE-02
F-UTL-04	<i>ApiResponse.success</i> respeta un código de estado personalizado	BE-02

4.8 Frontend — clases utilitarias (FE)

ID Caract.	Característica	Elementos ejercitados
F-FEU-01	<i>Storage</i> persiste y lee el token	FE-01
F-FEU-02	<i>Storage</i> persiste y lee el usuario	FE-01
F-FEU-03	<i>Storage.limpiarSesion</i> elimina tanto token como usuario	FE-01
F-FEU-04	<i>Storage.estaAutenticado</i> retorna false cuando no hay token	FE-01
F-FEU-05	<i>Auth.autenticado</i> delega en Storage	FE-02
F-FEU-06	<i>Auth.rol</i> retorna el rol del usuario actual	FE-02
F-FEU-07	<i>Auth.tieneRol(...roles)</i> coincide con uno de los roles	FE-02

4.9 Frontend — servicios sobre fetch (FE)

ID Caract.	Característica	Elementos ejercitados
F-FES-01	MesaService.listar llama a GET /mesas con token bearer	FE-09
F-FES-02	MesaService.crear llama a POST /mesas con cuerpo serializado	FE-09
F-FES-03	MesaService.actualizar llama a PUT /mesas/:id	FE-09
F-FES-04	MesaService.eliminar llama a DELETE /mesas/:id	FE-09
F-FES-05	MesaService.cambiarEstado llama a PATCH /mesas/:id/estado	FE-09
F-FES-06	El token bearer se omite del encabezado cuando no existe sesión	FE-09

4.10 Frontend — bus de eventos y helpers de UI (FE)

ID Caract.	Característica	Elementos ejercitados
F-FEE-01	EventBus.on/off/emit despacha eventos correctamente	FE-13
F-FEE-02	EventBus.off elimina un manejador específico	FE-13
F-FET-01	Toast renderiza un elemento con el nivel solicitado	FE-11
F-FEM-01	Modal.open/close alterna el elemento modal	FE-12

5. CARACTERÍSTICAS QUE NO SE PROBARÁN

Las siguientes se excluyen explícitamente de este plan de pruebas unitarias y se cubren en otros niveles de prueba:

1. **Integración con MongoDB:** Repositorios, modelos de Mongoose, persistencia: cubierto por un plan de pruebas de integración.
2. **Enrutamiento y orquestación HTTP:** Controladores de Express, flujo de petición/respuesta: plan de pruebas de integración / API.
3. **Flujos end-to-end:** Inicio de sesión, colocación completa de un pedido, de cocina a entrega, cierre de caja: plan de pruebas de sistema.
4. **Canales en tiempo real de Socket.IO:** cubiertos por el plan de pruebas de sistema.
5. **Flujo público por QR:** cubierto por el plan de pruebas de sistema.
6. **Requisitos no funcionales:** rendimiento, carga, escalabilidad, accesibilidad, pruebas de penetración de seguridad: cubiertos por planes dedicados.
7. **Seeders y scripts de migración de base de datos:** herramientas operativas, no son código del SUT.
8. **Renderizado HTML y maquetación visual:** cubiertos por el plan de pruebas de aceptación.

6. ENFOQUE DE PRUEBAS

6.1 Niveles de prueba

Este plan cubre únicamente pruebas unitarias (el nivel más bajo). La pirámide de pruebas del proyecto es:

REEMPLAZAR EN GOOGLE DOCS, SMARTART NO PUEDE SER IMPORTADO

Aceptación	← manual + E2E
Sistema	← E2E / Cypress o equivalente
Integración	← supertest + mongodb-memory-server
UNITARIA	← este plan (Jest / Vitest)

6.2 Técnica de prueba

Las técnicas de prueba utilizadas son:

Código	Técnica	Usada para
T1	Partición de equivalencia	reglas de validación, reglas de negocio
T2	Análisis de valores límite	límites numéricos (cantidad, capacidad, descuento, precio)
T3	Tabla de decisión	validación guiada por canal en PedidoValidator / PedidoBuilder
T4	Transición de estados	máquina de estados de bloqueo en AuthService.login
T5	Adivinación de errores	nulo, indefinido, cadena vacía, tipos incorrectos, entradas enormes
T6	Aislamiento basado en mocks	AuthService, MesaService, MesaService (HTTP)

6.3 Framework de pruebas

Capa	Framework	Runner	Librería de aserciones	Mocking
Backend	Jest	jest	Jest expect	Mocks de Jest / jest.mock()
Frontend	Vitest + jsdom	vitest	Vitest expect	vi.fn(), vi.stubGlobal

6.4 Convención de estructura de pruebas

backend/

src/

utils/

ApiError.js

ApiError.spec.js ← spec unitaria colocizada

validators/

MesaValidator.js

MesaValidator.spec.js

...

jest.config.cjs

package.json (añade scripts: "test", "test:unit", "test:coverage")

frontend/

utils/

storage.js

storage.spec.js

validators/

MesaValidator.spec.js

services/

mesaService.spec.js

vitest.config.js

`package.json` (añade scripts: "test", "test:unit", "test:coverage")

6.5 Patrón de diseño de pruebas

Todas las pruebas unitarias siguen el patrón Arrange / Act / Assert (AAA):

```
test("MesaValidator rechaza capacidad negativa", () => {  
  // Arrange  
  const datos = { numero: 5, capacidad: -1 };  
  
  // Act + Assert  
  expect(() => MesaValidator.validar(datos)).toThrow(ApiError);  
});
```

6.6 Estrategia de mocks

- **Servicios del backend:** el repositorio y bcrypt / jwt se sustituyen con mocks de Jest. La unidad bajo prueba es el servicio en sí.
- **Middlewares del backend:** las pruebas reciben objetos req / res simulados y un espía next.
- **Servicios del frontend:** global.fetch se reemplaza con vi.stubGlobal y manejadores vi.fn() por prueba.
- **Almacenamiento del frontend:** el localStorage de jsdom se reinicia entre pruebas.
- No se usan red real ni base de datos real.

6.7 Objetivos de cobertura de código

Capa	Objetivo
Utilidades backend	≥ 95% cobertura de líneas
Validadores backend	100% cobertura de líneas
Patrones backend	100% cobertura de líneas
Servicios backend	≥ 85% cobertura de líneas
Middleware backend	≥ 90% cobertura de líneas
Validadores frontend	100% cobertura de líneas
Utilidades frontend	≥ 90% cobertura de líneas
Servicios frontend	≥ 80% cobertura de líneas
Backend global	≥ 90% cobertura de líneas
Frontend global	≥ 85% cobertura de líneas

Una cobertura por debajo del objetivo requiere justificación explícita en el informe resumen de pruebas.

6.8 Gestión de datos de prueba

- Los fixtures de prueba se almacenan en carpetas `__fixtures__` / colocalizadas o se construyen inline en el spec.
- No se usan datos de producción. No se cargan seeders.
- Cada prueba reinicia sus mocks y el DOM en memoria.

6.9 Herramientas e Integración Continua

- Linting: ESLint con las reglas existentes del proyecto.

- Flujo de CI: lint → test:backend → test:frontend en cada Pull Request.
- El reporte de cobertura se publica como artefacto del CI.

7. CRITERIOS DE APROBACIÓN/FALLO

7.1 Aprobación/Fallo por caso de prueba

Un caso de prueba pasa cuando:

1. El resultado real coincide exactamente con el resultado esperado (o con un matcher explícito, p. ej. `toThrow(ApiError)`).
2. No se lanza ninguna excepción no manejada distinta a la afirmada.
3. Todas las aserciones dentro del cuerpo de la prueba se satisfacen.

Un caso de prueba falla cuando se incumple cualquiera de las condiciones anteriores.

7.2 Aprobación/Fallo por módulo

Un módulo pasa cuando:

- El 100% de sus casos de prueba pasan.
- La cobertura alcanza el objetivo por capa definido en §6.7.

Un módulo falla cuando cualquier caso de prueba falla o la cobertura cae por debajo del objetivo.

7.3 Aprobación/Fallo por componente (backend / frontend)

Un componente pasa cuando todos los módulos en alcance de dicho componente pasan.

7.4 Aprobación/Fallo global

La fase de pruebas unitarias se considera completa y aprobada cuando:

- Cada elemento en alcance de §3 tiene al menos un caso de prueba aprobado.
- Se cumplen los objetivos de cobertura de §6.7 o se renuncian formalmente.
- No hay defectos abiertos de severidad 1 (ver §13.2).

8. CRITERIOS DE SUSPENSIÓN Y REQUISITOS DE REANUDACIÓN

8.1 Criterios de suspensión (cuándo detener la fase)

La fase de pruebas unitarias se suspenderá si ocurre cualquiera de los siguientes eventos:

Código	Disparador
S-01	La compilación/carga del build bajo prueba falla (p. ej. errores de import en ESM).
S-02	Más del 20% de los casos de prueba de un mismo módulo fallan en la primera ejecución.

S-03	La cobertura del backend o frontend cae por debajo del 60% en un gate temprano.
S-04	Se descubre un defecto de severidad 1 en el código de producción que bloquea más de 5 tests.
S-05	Una dependencia externa crítica no está disponible (runner de CI, registro de npm).
S-06	El código bajo prueba se modifica mientras un ciclo de pruebas está en curso.

8.2 Requisitos de reanudación

Las pruebas solo se reanudarán cuando:

Código	Requisito
R-01	La compilación es limpia y todos los archivos spec cargan sin errores.
R-02	Existe una corrección documentada para cada defecto bloqueante, o el defecto fue diferido por el Líder de QA.
R-03	Se añade un plan de re-prueba al calendario (Sección 12).
R-04	Se reevalúa la línea base de cobertura tras la corrección.

9. ENTREGABLES DE LAS PRUEBAS

Entregable	Descripción
Plan de pruebas (este documento)	Plan de pruebas unitarias según IEEE 829
Especificaciones de diseño de prueba	Por cada elemento: entradas, salidas, necesidades del entorno (Apéndices A y B)
Especificaciones de casos de prueba	Catalogados en el Apéndice C
Especificaciones de procedimientos de prueba	Archivos *.spec.js — especificaciones ejecutables (Jest / Vitest)
Bitácoras de prueba	Salida del pipeline de CI + bitácoras por módulo
Reportes de anomalías de prueba	Por cada test fallido: id, resumen, entradas, esperado, real, severidad, entorno
Informe resumen de pruebas	Al cierre de la fase: conteos pasa/falla, cobertura, defectos, sign-off
Reporte de cobertura	HTML + Icov, publicado por el CI

Convención de nombres: TP-BOCATTO-UT-001 para este plan, TC-BOCATTO-UT-NNN para casos de prueba, TA-BOCATTO-UT-NNN para reportes de anomalía.

10. Tareas de Pruebas

ID	Tarea	Responsable	Depende de
T-01	Añadir Jest al backend, configurar ESM y entorno Node sin jsdom	Backend	—
T-02	Añadir Vitest + jsdom al frontend, configurar scripts	Frontend	—
T-03	Crear archivos spec para utilerías del backend (ApiError, ApiResponse)	Backend	T-01
T-04	Crear archivos spec para validadores del backend	Backend	T-01
T-05	Crear archivos spec para patrones del backend (Builder, Director, Prototype)	Backend	T-01
T-06	Crear archivos spec para DTOs y mappers del backend	Backend	T-01
T-07	Crear archivos spec para servicios del backend (repositorios mockeados)	Backend	T-01
T-08	Crear archivos spec para middlewares del backend	Backend	T-01
T-09	Crear archivos spec para utilerías del frontend (Storage, Auth, EstadoPedido)	Frontend	T-02
T-10	Crear archivos spec para validadores del frontend	Frontend	T-02
T-11	Crear archivos spec para servicios del frontend (fetch sustituido)	Frontend	T-02
T-12	Crear archivos spec para helpers de UI del frontend (Toast, Modal, EventBus)	Frontend	T-02
T-13	Cablear el flujo de CI: lint, test, cobertura	DevOps	T-01, T-02
T-14	Generar el reporte de cobertura y validar objetivos	Líder de QA	T-13
T-15	Producir el informe resumen de pruebas	Líder de QA	T-14

11. NECESIDADES DEL ENTORNO

11.1 Hardware

- Equipos de desarrollo: 1 CPU, 4 GB de RAM mínimo.
- Runner de CI: 2 CPU, 4 GB de RAM.

11.2 Software

Elemento	Backend	Frontend
Runtime	Node.js ≥ 20 LTS	Node.js ≥ 20 LTS (solo para el runner)
Framework de pruebas	Jest 29	Vitest 1.x
Emulación de DOM	no requerida	jsdom
Linting	ESLint (existente)	ESLint (existente)
Mocking	Mocks de Jest	vi.fn, vi.stubGlobal
Reporte de cobertura	c8 o Jest --coverage	@vitest/coverage-v8
CI	GitHub Actions	GitHub Actions

11.3 Red

- Las pruebas no deben requerir acceso a la red. fetch se sustituye; no hay llamadas a la API real.
- El registro de npm solo se permite en el momento de la instalación.

11.4 Archivos de configuración a añadir

- backend/jest.config.cjs — testEnvironment: "node", consciente de ESM, transform: {}.
- backend/package.json — "test": "jest", "test:coverage": "jest --coverage".
- frontend/vitest.config.js — environment: "jsdom".
- frontend/package.json — "test": "vitest run", "test:coverage": "vitest run --coverage".

12. CALENDARIO

El cronograma se expresa en días hábiles desde el inicio de la fase de pruebas unitarias (T0).

Fase	Días	Criterio de salida
K0 — Preparación del entorno de pruebas (T-01, T-02)	1	Todos los comandos de prueba corren con 0 tests
K1 — Utilidades y validadores del backend (T-03, T-04)	2	Todos los spec BE-01..08 en verde
K2 — Patrones del backend (T-05)	1	BE-11..15 en verde
K3 — DTO/mapper del backend (T-06)	1	BE-09, BE-10 en verde
K4 — Servicios y middleware del backend (T-07, T-08)	3	BE-16..20 en verde
K5 — Utilerías y validadores del frontend (T-09, T-10)	2	FE-01..08 en verde
K6 — Servicios y helpers de UI del frontend (T-11, T-12)	2	FE-09..13 en verde
K7 — Integración con CI (T-13)	1	Flujo de CI en verde
K8 — Validación de cobertura y sign-off (T-14, T-15)	1	Informe resumen de pruebas aprobado
Total	14	

Hitos:

- M1 fin de K0 — entorno listo.
- M2 fin de K4 — pruebas unitarias del backend completas.
- M3 fin de K6 — pruebas unitarias del frontend completas.
- M4 fin de K8 — fase de pruebas unitarias firmada.

13. RIESGOS Y CONTINGENCIAS

13.1 Riesgos del proyecto

ID	Riesgo	Probabilidad	Impacto	Mitigación
R-1	El acoplamiento fuerte de los servicios a los repositorios dificulta el mocking	Media	Medio	Usar fábricas de mocks de Jest; refactorizar servicios para inyectar repositorios (sin cambio de alcance)
R-2	La interoperabilidad ESM + Jest retrasa la configuración	Media	Bajo	Usar el soporte experimental ESM de Jest o babel-jest solo si es estrictamente necesario
R-3	Los servicios del frontend no están diseñados para ser testeables (sin DI)	Alta	Media	Sustituir global.fetch; tratar los servicios como caja negra por método
R-4	El comportamiento de los validadores diverge entre frontend y backend	Alta	Media	Crear pruebas espejo; documentar las diferencias intencionales
R-5	Los objetivos de cobertura se incumplen por presión de tiempo	Media	Media	Gate de cobertura en CI; las desviaciones requieren sign-off formal en el resumen
R-6	Cambios continuos en el código fuente durante la fase de pruebas	Media	Alto	Aplicar los criterios de suspensión de §8; fijar una etiqueta al inicio de la fase
R-7	Falta de tiempo o personal para crear todos los archivos spec	Media	Alto	Priorizar validadores y patrones; diferir los helpers de UI a una ola posterior

13.2 Esquema de severidad de defectos

Severidad	Definición
S1	Bloquea la fase de pruebas; el test no puede ejecutarse o revela lógica de negocio central incorrecta.
S2	El test falla; el defecto afecta una característica documentada, pero no bloquea la fase.
S3	Cosmético/menor; el test pasa con un workaround documentado.
S4	Solo documentación/nomenclatura/estilo.

14. APROBACIONES

Rol	Nombre	Firma	Fecha
Líder de QA			
Tech Lead BE			
Tech Lead FE			
Project Manager			

Sangolquí, a 09 de abril de 2026